組み込みモジュール設計パターン 教科書

Q&A で学ぶ 組み込みモジュール設計の基礎

バージョン	1.3
作成日	2026 年 4 月 12 日
著者	Shiozawa

本書は、Arduino フレームワーク対応マイコン全般で使える
「組み込みモジュール設計パターン」を Q&A 形式で解説する教科書です。

コア設計は ESP32 に限らず Arduino 互換のあらゆるボードに適用でき、
クラスの基礎から 3 フェーズ実行モデルまで段階的に学べます。

対象読者と読み方

「C++ のクラスはなんとなく知ってるけど、
部品化の経験がない」方を主な対象としています。

第 1 章「前提知識」ではクラスの基礎を解説しています。

構造体・クラス・コンストラクタを既にご存知の方は、
第 1 章を飛ばして第 2 章から読み始めてください。

本リポジトリのドキュメント構成

教科書（本書）→ **実装ガイド**（`doc-guide/`）→ **設計仕様書**（`doc/`）
の順に読み進めることを推奨します。

改訂履歴

版	日付	著者	変更内容
1.0	2026-03-14	Shiozawa	初版作成。C++ クラスの基礎から 3 フェーズモデルまでを Q&A 形式で解説
1.1	2026-03-17	Shiozawa	関連ドキュメントの相互参照を追加。日付・Rev 番号を全ドキュメントと同期
1.2	2026-03-18	Shiozawa	ESP32 固有の記述を Arduino/組み込み汎用に変更。対象読者を明確化
1.3	2026-04-12	Shiozawa	改訂履歴セクションを追加。バージョン表記を他ドキュメントと同期

目次

改訂履歴 1

1 前提知識 — C++ クラスの基礎 3
- 1.1 変数 — 値の箱 . . . 3
- 1.2 配列 — 箱の列 . . . 3
- 1.3 関数 — 処理の箱 . . . 3
- 1.4 構造体 — 変数の箱 . . . 4
- 1.5 クラス — 変数と関数の箱 . . . 4

2 全部 main.cpp に書くとどうなる？ 9
- 2.1 問題 1: 肥大化 . . . 10
- 2.2 問題 2: マジックナンバー . . . 10
- 2.3 問題 3: 再利用できない . . . 11

3 モジュールという考え方 11

4 インターフェースで統一する 14
- 4.1 virtual とは . . . 15
- 4.2 = 0（純粋仮想関数）とは . . . 15
- 4.3 配列で回せるメリット . . . 15

5 データの流れを整理する 16
- 5.1 なぜ必要なの？ . . . 18

6 3 フェーズモデル 19

7 プロジェクト構成を理解する 22
- 7.1 コア層: ModuleCore/ . . . 23
- 7.2 個別モジュール: XxxModule/ . . . 23

8 実コードを読んでみよう 26
- 8.1 コンストラクタ . . . 27
- 8.2 init() . . . 27
- 8.3 updateOutput() . . . 27

9 実践 — センサーモジュールを作る 28
- 9.1 ステップ 1: Config 構造体 . . . 29
- 9.2 ステップ 2: Data 構造体 . . . 29
- 9.3 ステップ 3: クラス宣言（.h ファイル） . . . 29
- 9.4 ステップ 4: メソッド実装（.cpp ファイル） . . . 30
- 9.5 ステップ 5: SystemData に追加 . . . 30
- 9.6 ステップ 6: ProjectConfig.h に追加 . . . 30
- 9.7 ステップ 7: main.cpp に追加 . . . 31

10 設計のコツ集 32
10.1 「No such file or directory」 35
10.2 「does not name a type」 35
10.3 「is not a member of 'SystemData'」 35
10.4 「marked 'override', but does not override」 36

付録 A: PlatformIO のススメ 36

1 前提知識 — C++ クラスの基礎

この章では、本書を読むために必要な C++ クラスの基礎知識を解説します。

▷ すでにクラスを理解している方へ

構造体・クラス・コンストラクタ・アクセス修飾子を理解している方は、この章を飛ばして次の「全部 main.cpp に書くとどうなる？」に進んでください。

Q: プログラミングの部品を整理しよう

C++ にはいろいろな「部品」が出てきますが、実はどれも「何かを入れる箱」としてイメージできます。まずは全体像をつかみましょう。

1.1 変数 — 値の箱

一番シンプルな部品です。1 つの値を入れておく箱。

```
int speed = 100;        // 整数を1つ入れる箱
float temperature = 25.3; // 小数を1つ入れる箱
bool isRunning = true; // true/falseを1つ入れる箱
```

1.2 配列 — 箱の列

同じ種類の箱を一列に並べたもの。番号（インデックス）で「何番目の箱か」を指定してアクセスします。

```
int scores[3] = {80, 95, 70};  // int型の箱が3つ並んだ列
scores[0];  // → 80（0番目）
scores[1];  // → 95（1番目）
```

1.3 関数 — 処理の箱

「一連の処理」をまとめて名前を付けた箱です。呼び出すと中の処理が実行されます。

```
// 関数の定義（処理を箱に入れる）
void blinkLed(int pin) {
    digitalWrite(pin, HIGH);
    delay(500);
    digitalWrite(pin, LOW);
}
```

```
// 関数の呼び出し（箱を使う）
blinkLed(13);  // ピン13でLEDを点滅
```

1.4 構造体 — 変数の箱

関連する変数をまとめて 1 つの「箱」にしたもの。中に複数の変数を持てます。

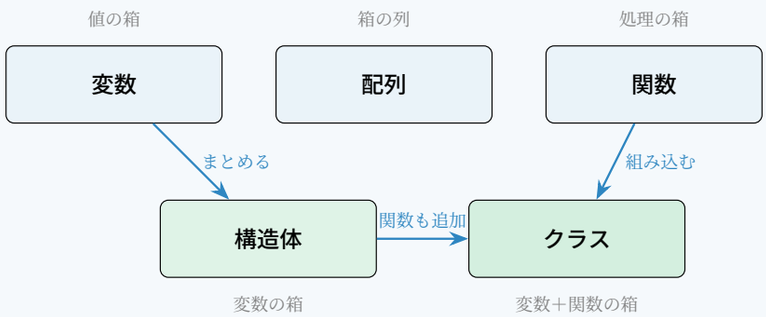
```
struct LedData {            // 「LedDataという箱の設計図」
    bool state = false;     //   中に入る変数1
    uint8_t brightness = 0; //   中に入る変数2
};
```

1.5 クラス — 変数と関数の箱

構造体の進化版。変数（データ）だけでなく、関数（操作）も一緒にまとめた箱です。

```
class LedModule {           // 「LedModuleという箱の設計図」
    uint8_t _pin;           //   中に入る変数（データ）
    bool _state;            //   中に入る変数（データ）
    void turnOn();          //   中に入る関数（操作）
    void turnOff();         //   中に入る関数（操作）
};
```

ここまでの関係をまとめると、こうなります。



この先の章では、構造体とクラスを詳しく見ていきます。

Q: 構造体（struct）って何？

プログラムを書いていると、「関連する変数をセットで扱いたい」場面がよくあります。
たとえば、LED の状態を管理する変数が 2 つあるとします。

× バラバラの変数で管理

```
bool ledState = false;
uint8_t ledBrightness = 0;

bool led2State = false;
```

```
uint8_t led2Brightness = 0;
// LED が増えるたびに変数が倍増...
```

LED が増えるたびに変数名を考えなくてはいけませんし、「どの変数がどの LED のものか」が分かりにくくなります。
そこで「構造体」の出番です。関連する変数を 1 つの型としてまとめられます。

✓ 構造体でまとめる

```
struct LedData {
    bool state = false;
    uint8_t brightness = 0;
};

LedData led1;  // LED1のデーター式
LedData led2;  // LED2のデーター式

led1.state = true;        // ドット(.)でアクセス
led2.brightness = 128;
```

構造体を使えば、「LED に必要なデータは何か」が型として明確になり、LED が何個増えても変数の定義は 1 行で済みます。

! 構造体の定義だけでは「実体」はない

初学者がつまずきやすいポイントです。`struct LedData {...};` と書いた時点では、メモリ上に何も作られていません。これは「設計図（型）」を定義しただけです。

```
// これは「設計図」を書いただけ（実体はまだない）
struct LedData {
    bool state = false;
    uint8_t brightness = 0;
};

// ここで初めて「実体（インスタンス）」が作られる
LedData led1;   // LedData型の実体を1つ作成
LedData led2;   // もう1つ作成（led1とled2は別々の実体）
```

たとえるなら、構造体の定義は「たい焼きの型（金型）」です。型を作っただけではたい焼きは食べられません。型に生地を流し込んで焼いて初めて、食べられるたい焼き（実体）ができます。

- **型（設計図）**：`struct LedData {...};` — 「こういうデータの組み合わせがある」と定義
- **実体（インスタンス）**：`LedData led1;` — 実際にメモリ上に作られた「もの」

1 つの型から何個でも実体を作れます。led1 と led2 はどちらも LedData 型ですが、それぞれ独立した別のデータを持ちます。

Q: 構造体とクラスって何が違うの？

構造体は「データをまとめる」ものでした。クラスはさらに「そのデータを操作する関数」も一緒にまとめます。

✓ クラス ＝ データ ＋ 関数

```
class LedModule {
private:
    uint8_t _pin;        // データ（メンバ変数）
    bool _state;

public:
    // コンストラクタ（後述）
    LedModule(uint8_t pin) : _pin(pin), _state(false) {}

    // メンバ関数
    void turnOn() {
        _state = true;
        digitalWrite(_pin, HIGH);
    }

    void turnOff() {
        _state = false;
        digitalWrite(_pin, LOW);
    }
};
```

構造体でもメンバ関数を持てますが、C++ の慣習として次のように使い分けます。

- **struct** — データの入れ物（Config、Data など）
- **class** — データ＋操作をセットにしたもの（モジュールなど）

▷ struct と class の技術的な違い

C++ では struct と class の唯一の違いは「デフォルトのアクセス修飾子」です。struct はデフォルト public、class はデフォルト private です。本書ではデータの入れ物に struct、操作を持つものに class を使います。

！ クラスも「型を定義している」だけ

構造体と同じく、クラスの定義だけでは実体は作られません。

```
// これは設計図（型）
class LedModule {
    uint8_t _pin;
    bool _state;
public:
    LedModule(uint8_t pin) : _pin(pin), _state(false) {}
    void turnOn();
    void turnOff();
```

```
};

// ここで実体（インスタンス）が作られる
LedModule led1(13);   // ピン13のLEDモジュール
LedModule led2(14);   // ピン14のLEDモジュール（別の実体）
```

led1.turnOn() と呼ぶと led1 だけが点灯し、led2 には影響しません。同じ設計図から作られていても、実体はそれぞれ独立しています。
この「型の定義」と「実体の作成」が別であるという点は、本書の設計パターン全体を通じて重要な考え方です。Config 構造体の「型定義」と Config インスタンスの「値定義」が別ファイルになっているのも、この原則に基づいています。

Q: コンストラクタって何？

コンストラクタは「オブジェクトが作られるときに自動で呼ばれる関数」です。初期化忘れを防げます。

× 初期化忘れの危険

```
LedModule led;          // ピン番号が未設定！
led.turnOn();           // どのピンが光る？→バグ
```

✓ コンストラクタで確実に初期化

```
class LedModule {
private:
    uint8_t _pin;
    bool _state;

public:
    // コンストラクタ — 初期化子リストで初期化
    LedModule(uint8_t pin) : _pin(pin), _state(false) {
        // ピンを指定しないと作れない
    }
};

LedModule led(13);   // OK: ピン13で作成
// LedModule led2;   // コンパイルエラー！ピン未指定
```

コロン（:）の後に書く _pin(pin), _state(false) の部分を「初期化子リスト」と呼びます。メンバ変数を効率よく初期化する書き方です。

Q: private/public って何？

クラスの中身を「外から触れる部分」と「触れない部分」に分けるしくみです。

イメージとしては、銀行の窓口を思い浮かべてください。

- **public**（窓口） — お客さん（外部のコード）がアクセスできる
- **private**（金庫） — 銀行員（クラス内部）だけがアクセスできる

```
class LedModule {
private:
    uint8_t _pin;     // 外から直接変更させない

public:
    LedModule(uint8_t pin) : _pin(pin) {}
    void turnOn();    // 外から呼べる
};

LedModule led(13);
led.turnOn();       // OK: publicなので呼べる
// led._pin = 99;   // コンパイルエラー！privateには触れない
```

なぜわざわざ隠すのか？ 外部から _pin を勝手に変更されると、LED モジュールの動作が壊れる可能性があるからです。「触ってほしくないものは private にする」のが安全設計の基本です。

▷ 命名の慣習

private メンバ変数には先頭にアンダースコア（_）を付ける慣習があります。_pin、_state のように書くと、「これは内部変数だな」と一目で分かります。

Q: virtual って何？（予告）

virtual は「クラスの継承」という仕組みで使うキーワードです。これを使うと、異なるクラスを同じ型として扱えるようになります。
……と言われてもピンと来ないと思います。
大丈夫です。virtual の詳しい説明は本編の「インターフェースで統一する」の章で、具体例を交えて解説します。ここでは「そういうキーワードがあるんだな」とだけ覚えておいてください。

▷ ここまでのまとめ

- **構造体** — 関連する変数をまとめる箱
- **クラス** — データ＋操作をセットにしたもの
- **コンストラクタ** — オブジェクト生成時に自動実行される初期化関数
- **private/public** — アクセス範囲を制限して安全性を確保
- **virtual** — 「インターフェースで統一する」の章で詳しく！

これだけ分かっていれば、本編を読み進められます。さあ、次の章に進みましょう！

2 全部 main.cpp に書くとどうなる？

▷ C++ クラスの復習が必要な方へ

この章から C++ のクラスを使います。構造体・クラス・コンストラクタに自信がある方はそのまま読み進めてください。不安な方は第 1 章「前提知識 — C++ クラスの基礎」（3 ページ）で復習できます。

▷ 関連ドキュメントとプロジェクト構成

本リポジトリには 2 つのプロジェクトと 3 つのドキュメントがあります。

プロジェクト:

- `sample/` — 設計パターンの構造説明用サンプル（動作未確認）
- `verified/` — 実機で動作確認済みのプロジェクト

ドキュメント:

- **教科書**（本書） — パターンの段階的な学習（「なぜそうするか」）
- **実装ガイド**（`doc-guide/`） — `sample/`の実コードを題材にした解説（「実際にどう書いたか」）
- **設計仕様書**（`doc/`） — ルール集・API 仕様（「何を守るか」）

本書で基礎を学んだら、実装ガイドで実コードの理解を深めてください。設計仕様書はルール集として必要なときに参照する使い方を推奨します。

! サンプルコードについて

`sample/` に実装されているモジュール群のコードは、あくまで**設計パターンを具体的に示すためのサンプル実装**である。実機での動作検証は行っていないため、実プロジェクトへの適用時は十分なテストを実施すること。動作確認済みのコードは `verified/` を参照すること。

Q: 全部 main.cpp に書くの、何がまずいの？

まずは「よくある書き方」を見てみましょう。LED2 個と温度センサー 1 個を制御するプログラムです。

× 全部 main.cpp にベタ書き

```
#include <Arduino.h>

// ピン番号（マジックナンバー）
#define LED1_PIN 2
#define LED2_PIN 4
#define TEMP_PIN 34

// グローバル変数がバラバラに並ぶ
bool led1State = false;
bool led2State = false;
float temperature = 0.0;
```

```
unsigned long lastBlink = 0;
unsigned long lastRead = 0;

void setup() {
    Serial.begin(115200);
    pinMode(LED1_PIN, OUTPUT);
    pinMode(LED2_PIN, OUTPUT);
    // 温度センサーはアナログなのでpinMode不要
}

void loop() {
    unsigned long now = millis();

    // LED1: 500msごとに点滅
    if (now - lastBlink >= 500) {
        lastBlink = now;
        led1State = !led1State;
        digitalWrite(LED1_PIN, led1State ? HIGH : LOW);
    }

    // LED2: 常時点灯
    digitalWrite(LED2_PIN, HIGH);

    // 温度センサー: 1秒ごとに読み取り
    if (now - lastRead >= 1000) {
        lastRead = now;
        int raw = analogRead(TEMP_PIN);
        temperature = raw * 3.3 / 4095.0 * 100.0;
        Serial.print("Temp: ");
        Serial.println(temperature);
    }

    // ここにモーター制御を追加したら？
    // さらにディスプレイ表示を追加したら？
    // ...どんどん膨れ上がる
}
```

動くことは動きます。でも、この書き方には 3 つの大きな問題があります。

2.1 問題 1: 肥大化

今は約 40 行ですが、モーター制御、ディスプレイ表示、通信機能……と追加していくと、あっという間に数百行になります。「LED1 の点滅ロジックはどこだっけ？」と探すだけで一苦労です。

2.2 問題 2: マジックナンバー

`LED1_PIN`、`500`（点滅間隔）、`3.3 / 4095.0 * 100.0`（変換式）といった数値がコード中に散らばっています。後から見て「この数字は何？」となりがちです。

2.3 問題 3: 再利用できない

別のプロジェクトでも LED 点滅が必要になったとき、このコードから LED 関連の部分だけを抜き出すのは大変です。変数名が被ったり、依存関係が絡み合っていたりします。

Q: この教科書で何を学ぶの？

この教科書では、上のような「ベタ書きコード」を段階的に改善していきます。

1. **第 3 章: モジュールという考え方** — 1 つのハードウェアを 1 つのクラスに分離する
2. **第 4 章: インターフェースで統一する** — 異なるモジュールを同じ方法で扱う
3. **第 5 章: データの流れを整理する** — モジュール間のデータ受け渡しを安全にする
4. **第 6 章: 3 フェーズモデル** — loop() の処理を「入力→ロジック→出力」に整理する
5. **第 7 章: プロジェクト構成を理解する** — フォルダ配置と PlatformIO の使い方
6. **第 8 章: 実コードを読んでみよう** — LedModule の実コードを読み解く
7. **第 9 章: 実践** — 温度センサーモジュールを一から作る
8. **第 10 章: 設計のコツ集** — よくあるエラーと実務で役立つ Tips

最終的に、先ほどのベタ書きコードがどう変わるか？ それは 3 フェーズモデルの章で明らかになります。

3 モジュールという考え方

Q: モジュールって何？

モジュールとは、「1 つのハードウェア部品の制御を、1 つのクラスにまとめたもの」です。
ルールはシンプルです。

1 ハードウェア ＝ 1 クラス（1 モジュール）

第 1 章のベタ書きコードから、LED 部分だけを抜き出してクラスにしてみましょう。

✓ **LedModule を抽出**

```
// LedModule.h
class LedModule {
private:
    uint8_t _pin;
    bool _state;

public:
    LedModule(uint8_t pin) : _pin(pin), _state(false) {}

    bool init() {
        pinMode(_pin, OUTPUT);
        return true;
    }

```

```
    void turnOn() {
        _state = true;
        digitalWrite(_pin, HIGH);
    }

    void turnOff() {
        _state = false;
        digitalWrite(_pin, LOW);
    }

    void toggle() {
        _state ? turnOff() : turnOn();
    }
};
```

これで main.cpp は次のようにすっきりします。

```
LedModule led1(2);   // ピン2のLED
LedModule led2(4);   // ピン4のLED

void setup() {
    led1.init();
    led2.init();
}
```

LED の制御に関するコードは `LedModule` の中に閉じ込められました。main.cpp を見ても「LED が 2 つあって、init している」ということが一目で分かります。

Q: なんで Config 構造体が必要なの？

先ほどの例では、コンストラクタにピン番号だけ渡しました。でも設定項目が増えるとどうなるでしょう？

× 引数が増えすぎる

```
// ピン番号、初期状態、PWMチャンネル、PWM周波数...
LedModule led(2, false, 0, 5000);
// どの引数が何を意味するか分からない！
```

✓ Config 構造体にまとめる

```
// 設定をまとめた構造体
struct LedConfig {
    uint8_t pin;
};

class LedModule {
private:
    LedConfig _config;  // 設定を丸ごと保持

```

```
public:
    LedModule(const LedConfig& config) : _config(config) {}

    bool init() {
        pinMode(_config.pin, OUTPUT);
        return true;
    }
};

// 使う側 — 何の設定か一目瞭然
const LedConfig LED1_CONFIG = { .pin = 2 };
LedModule led1(LED1_CONFIG);
```

Config 構造体を使うメリットは 3 つです。

1. 設定項目が増えても、構造体にフィールドを追加するだけ
2. 設定値を 1 か所（ProjectConfig.h）にまとめて管理できる
3. コンストラクタの引数の意味が明確

Q: Config と Data の違いは？

モジュールが扱う情報は、大きく 2 種類に分かれます。

	Config	Data
意味	起動時の設定	実行中に変化するデータ
いつ決まる？	プログラム開始前（コンパイル時）	プログラム実行中
変わる？	基本的に変わらない	常に変わる
例	ピン番号、I2C アドレス	LED 状態、センサー値

```
// Config — 起動時に決まる設定
struct LedConfig {
    uint8_t pin;        // ピン番号（変わらない）
};

// Data — 実行中に変わるデータ
struct LedData {
    bool state = false;         // ON/OFF状態
    uint8_t brightness = 0;     // 明るさ
};
```

▷ なぜ分けるの？

Config と Data を分けることで、「この値は実行中に変わるのか？」が型レベルで明確になります。Config は const にできるので、うっかり書き換えてしまうバグも防げます。

4 インターフェースで統一する

Q: モジュールが増えたらどうするの？

LED、温度センサー、モーター……とモジュールが増えていくと、こんなコードになりがちです。

× 呼び方がバラバラ

```
void setup() {
    led.init();
    tempSensor.begin();        // beginだったり
    motor.start();             // startだったり
    display.initialize();      // initializeだったり
}

void loop() {
    led.toggle();
    tempSensor.readValue();    // メソッド名もバラバラ
    motor.drive();
    display.refresh();
}
```

モジュールごとに初期化メソッドの名前が違う、更新メソッドの名前が違う。これでは新しいモジュールを追加するたびに「このモジュールは何を呼べばいいんだっけ？」と調べる羽目になります。
もっと困るのは、「全モジュールの初期化を順番に実行する」ような共通処理が書けないことです。

Q: IModule って何をしてくれるの？

IModule は「すべてのモジュールが守るべきルール」を定めたインターフェースです。

✓ IModule インターフェース

```
struct SystemData;  // 前方宣言（次の章で詳しく説明）

class IModule {
public:
    bool enabled = true;

    virtual bool init() = 0;
    virtual void updateInput(SystemData& data) {}
    virtual void updateOutput(SystemData& data) {}
    virtual void deinit() {}
    virtual ~IModule() {}
};
```

ここで出てくる virtual と = 0 について説明しましょう。

4.1 virtual とは

virtual は「この関数は子クラスで上書き（オーバーライド）できますよ」という印です。

4.2 = 0（純粋仮想関数）とは

= 0 は「この関数は子クラスで必ず実装してね。実装しないとコンパイルエラーにするよ」という意味です。IModule では init() だけが = 0 なので、必ず実装が必要です。
一方、updateInput() や updateOutput() は {}（空の中括弧）で「何もしないデフォルト実装」が書かれています。入力モジュールは updateInput() を、出力モジュールは updateOutput() をオーバーライドします。両方を持つモジュール（BLE 等）は両方をオーバーライドします。

4.3 配列で回せるメリット

すべてのモジュールが IModule を継承すると、こんなことができます。

✓ 配列でまとめて初期化

```
// 異なるクラスでも、IModule*の配列に入れられる
IModule* modules[] = {
    &led,
    &tempSensor,
    &motor,
};

// まとめて初期化
for (int i = 0; i < 3; i++) {
    if (!modules[i]->init()) {
        Serial.println("init failed!");
        modules[i]->enabled = false;
    }
}
```

更新は入力モジュールと出力モジュールで配列を分け、それぞれ updateInput() と updateOutput() を呼びます。この仕組みは第 5 章「3 フェーズモデル」で詳しく解説します。
LedModule も TempSensorModule も MotorModule も、すべて IModule*として統一的に扱えます。新しいモジュールを追加しても、for 文はそのままです。

▷ override キーワード

子クラスで親の仮想関数を実装するときは、override キーワードを付けましょう。タイプミスで別の関数を定義してしまうバグを防げます。

```
class LedModule : public IModule {
public:
    bool init() override { /* ... */ }       // OK
    void updateOutput(SystemData& data) override; // OK
};
```

Q: なんで enabled があるの？

ハードウェアは壊れることがあります。センサーが接触不良で応答しない、ディスプレイの初期化に失敗した……そんなとき、プログラム全体を止めたくはありません。
enabled フラグを使えば、初期化に失敗したモジュールだけをスキップできます。

```
for (int i = 0; i < MODULE_COUNT; i++) {
    if (!modules[i]->init()) {
        Serial.println("init failed, disabling");
        modules[i]->enabled = false;  // このモジュールだけ無効化
    }
}

void loop() {
    for (int i = 0; i < INPUT_COUNT; i++) {
        if (inputModules[i]->enabled) {    // 無効なモジュールはスキップ
            inputModules[i]->updateInput(systemData);
        }
    }
    // ... 出力モジュールも同様にenabledチェック
}
```

温度センサーが壊れても、LED やモーターは動き続けます。組み込みシステムでは「部分的に動く」ことが重要です。

5 データの流れを整理する

Q: モジュール同士が直接話すとなぜダメ？

「温度が 30 度を超えたら LED を点灯」したいとき、こんなコードを書きたくなるかもしれません。

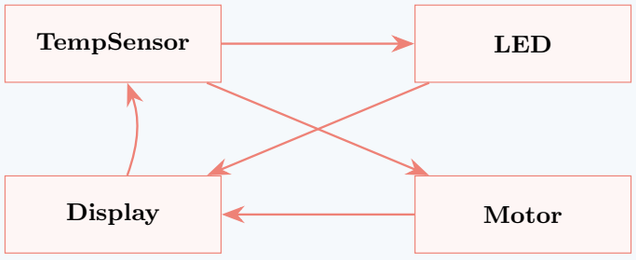
× モジュール同士が直接参照

```
class TempSensorModule {
private:
    LedModule* _led;  // LEDモジュールを直接持っている！

public:
    void run() {
        float temp = readSensor();
        if (temp > 30.0) {
            _led->turnOn();  // 直接操作
        }
    }
};
```

一見うまく動きそうですが、以下の問題が起きます。

- 温度センサーモジュールが LED モジュールに依存 → LED なしでは使えない
- 「温度でモーターも回したい」→ MotorModule*も追加？ → 依存がどんどん増える
- モジュール間の依存が複雑に絡み合い、変更が困難に（スパゲッティ化）



矢印（依存関係）が蜘蛛の巣のように広がっています。これでは 1 つのモジュールを変更すると、他のモジュールにも影響が波及します。

Q: SystemData って何？

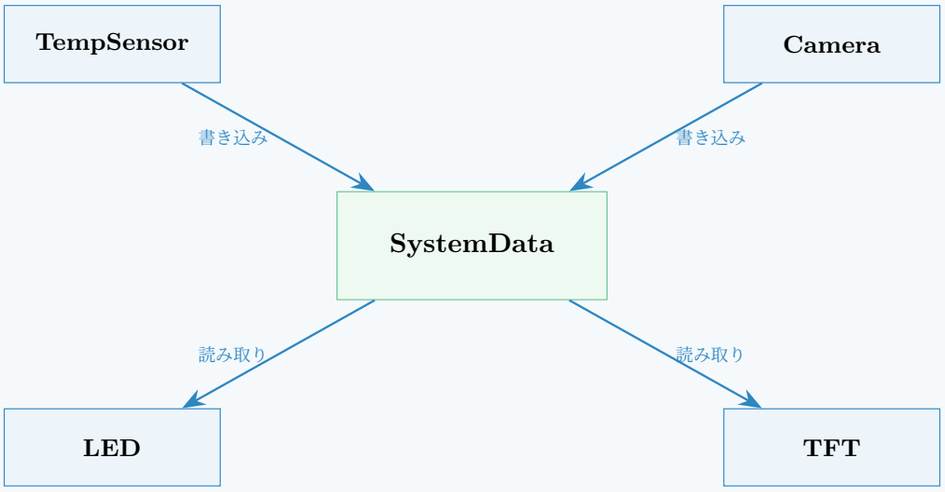
`SystemData` は、すべてのモジュールのデータを 1 か所にまとめた「伝言板」です。

✓ SystemData — モジュールの伝言板

```
#include "LedModule.h"
#include "TempSensorModule.h"

struct SystemData {
    LedData led;              // LEDの状態
    TempSensorData temp;      // 温度センサーの値
};
```

モジュール同士は直接会話しません。代わりに SystemData を介してデータをやり取りします。



- 入力モジュール（TempSensor 等）は SystemData にデータを書き込む
- 出力モジュール（LED 等）は SystemData からデータを読み取る

- モジュール同士は直接参照しない → 依存関係がシンプル

温度センサーは LED の存在を知りません。ただ SystemData の `temp` に値を書くだけです。LED をモーターに差し替えても、温度センサーのコードは 1 行も変わりません。

Q: 前方宣言って何？

IModule.h の中にこんな 1 行がありました。

```
struct SystemData;  // 前方宣言
```

これは「SystemData という型がどこかで定義されてますよ」とコンパイラに教える宣言です。

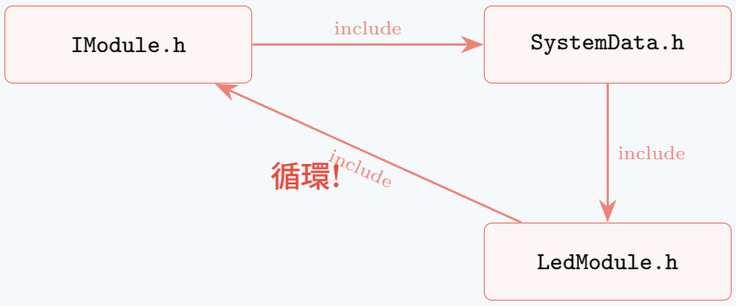
5.1 なぜ必要なの？

IModule.h は `updateInput(SystemData& data)` や `updateOutput(SystemData& data)` を宣言していますが、SystemData の中身を知る必要はありません（参照で受け取るだけなので）。
もし IModule.h が `SystemData.h` をインクルードすると、こんな問題が起きます。

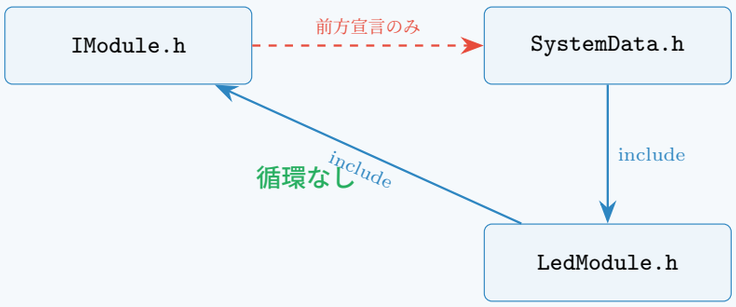
× 循環依存

```
// IModule.h が SystemData.h をインクルード
// SystemData.h が LedModule.h をインクルード
// LedModule.h が IModule.h をインクルード
// → IModule.h が SystemData.h をインクルード → 無限ループ！
```

図にすると、矢印がぐるっと一周しているのが分かります。



前方宣言を使えば、IModule.h から SystemData.h へのインクルードが不要になり、循環を断ち切れます。



赤い破線の部分が「前方宣言で置き換えた箇所」です。IModule.h は SystemData.h をインクルードしないので、循環が発生しません。

✓ 前方宣言で循環を回避

```
// IModule.h — SystemData.hをインクルードしない
struct SystemData;  // 「こういう型がある」とだけ宣言

class IModule {
public:
    virtual void updateInput(SystemData& data) {}
    virtual void updateOutput(SystemData& data) {}
    // 参照で受け取るだけなので、中身を知らなくてOK
};
```

▷ ヘッダーと cpp の使い分け

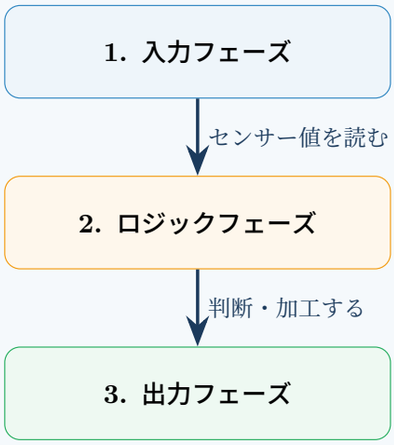
- **.h ファイル** — `struct SystemData;` の前方宣言のみ
- **.cpp ファイル** — `#include "SystemData.h"` でフル定義を使う

.cpp では実際に SystemData のメンバにアクセスするので、フル定義が必要です。このルールを守れば循環依存は起きません。

6 3 フェーズモデル

Q: loop() の中身はどう整理するの？

ここまでで、モジュール・インターフェース・SystemData の 3 つの道具が揃いました。最後のピースは「loop() の実行順序」です。
loop() の中身を 3 つのフェーズに分けます。これを「3 フェーズモデル」と呼びます。



工場の流れ作業に例えると分かりやすいです。

1. **入力フェーズ**（検品係） — センサーからデータを読み取り、SystemData に書き込む
2. **ロジックフェーズ**（判断係） — SystemData を見て、どう動くか判断する

3. **出力フェーズ**（作業係） — SystemData の指示に従い、LED やモーターを制御する

✓ 3 フェーズの loop()

```
// モジュール配列
IModule* inputModules[] = { &tempSensor };
IModule* outputModules[] = { &led };
const int INPUT_COUNT = 1;
const int OUTPUT_COUNT = 1;

void loop() {
    // 1. 入力フェーズ — センサー値をSystemDataに書き込む
    for (int i = 0; i < INPUT_COUNT; i++) {
        if (inputModules[i]->enabled) {
            inputModules[i]->updateInput(systemData);
        }
    }

    // 2. ロジックフェーズ — データを見て判断する
    applyPattern(systemData);

    // 3. 出力フェーズ — 判断結果に従い制御する
    for (int i = 0; i < OUTPUT_COUNT; i++) {
        if (outputModules[i]->enabled) {
            outputModules[i]->updateOutput(systemData);
        }
    }
}
```

Q: なんで順番が大事なの？

「入力→ロジック→出力」の順番を守らないと、思わぬバグが発生します。

× 出力が先に実行される場合

```
void loop() {
    // 出力が先！
    led.updateOutput(systemData);    // 前回のデータで光ってしまう

    // 入力が後
    tempSensor.updateInput(systemData);  // 今回のデータを読む

    applyPattern(systemData);  // 今回のデータで判断するが...
    // → LEDにはもう反映されない（次のloopまで待つ）
}
```

出力が先だと、LED は常に「1 ループ前のデータ」で動作します。温度が急上昇しても、LED の反応が 1 ループ分遅れます。通常は気にならない程度ですが、リアルタイム性が求められるシステムでは致命的です。また、ロジックフェーズをスキップして入力モジュールが出力モジュールのデータを直接書き換えると、「なぜこの LED が光っているのか」の理由がコード上で追えなくなります。

！クロス参照の禁止

入力モジュールが出力モジュールのデータを読み書きしたり、その逆をしたりすることを「クロス参照」と呼びます。これは禁止です。

- 入力モジュール → 自分の Data に書き込むだけ
- 出力モジュール → 自分の Data を読み取るだけ
- データの受け渡し → ロジックフェーズが担当

Q: ロジックフェーズでは何を書くの？

ロジックフェーズは「入力データを見て、出力データを決める」場所です。

✓ applyPattern() の例

```
void applyPattern(SystemData& data) {
    // 温度が30度を超えたらLEDを点灯
    if (data.temp.temperature > 30.0) {
        data.led.state = true;
    } else {
        data.led.state = false;
    }
}
```

ポイントは以下の通りです。

- ハードウェアを直接操作しない（`digitalWrite` 等を呼ばない）
- SystemData の値を読んで、SystemData の値を書き換えるだけ
- 「なぜ LED が光るのか」のルールがこの関数に集約される

ロジックが複雑になっても、この関数を読めば動作の全体像が分かります。

▷ 第 1 章のコードがこう変わった

第 2 章では約 40 行のベタ書きでしたが、3 フェーズモデルで整理すると次のような構成になります。

- `LedModule` — LED の制御（約 20 行）
- `TempSensorModule` — 温度の読み取り（約 20 行）
- `applyPattern()` — ルール（約 5 行）
- `main.cpp` — 配列定義 ＋ 3 フェーズループ（約 30 行）

合計行数は増えますが、各ファイルが短く、役割が明確です。

7 プロジェクト構成を理解する

ここまでで設計パターンの理論は一通り学びました。この章では、実際のリポジトリがどんなフォルダ構成になっているのかを見ていきます。

Q: リポジトリ全体の構成はどうなってるの？

本リポジトリには 2 つのプロジェクトが含まれています。

```
repository-root/
  |-- sample/              ... サンプルプロジェクト（設計パターンの構造説明
      用）
  |-- verified/            ... 動作確認済みプロジェクト（実機テスト済み）
  |-- doc/                 ... 設計仕様書
  |-- doc-textbook/        ... 教科書（本書）
  `-- doc-guide/           ... 実装ガイド
```

ディレクトリ	説明
sample/	設計パターンを具体的なコードで示すためのサンプル。動作未確認
verified/	実機で動作確認を行ったプロジェクト。フォルダごとダウンロードすればそのまま使える

各プロジェクトは完全に自己完結した PlatformIO プロジェクトです。フォルダ単体でビルド・書き込みができるようになっています。

Q: プロジェクトのフォルダ構成はどうなってるの？

各プロジェクト（sample/、verified/）は PlatformIO プロジェクトとして構成されています。ツリー構造を見てみましょう。

```
sample/  (or verified/)
  |-- platformio.ini       ... ビルド設定（ボード・ライブラリ等）
  |-- src/
  |     `-- main.cpp       ... エントリーポイント（setup / loop）
  |-- include/
  |     |-- SystemData.h   ... 全モジュールのDataを集約する構造体
  |     `-- ProjectConfig.h... 全モジュールのConfigインスタンス
  |-- lib/
  |     |-- ModuleCore/    ... コア層（プロジェクト非依存）
  |     |     |-- IModule.h
  |     |     `-- ModuleTimer.h
  |     |-- LedModule/     ... 個別モジュール
  |     |     |-- LedModule.h
  |     |     `-- LedModule.cpp
  |     |-- BuzzerModule/
  |     |     |-- BuzzerModule.h
```

```
|      |      `-- BuzzerModule.cpp
|      `-- ...                ... 他のモジュールも同じ構成
`-- test/                     ... テストコード置き場
```

大きく分けると 4 つのエリアがあります。

場所	役割
src/	エントリーポイント。main.cpp のみ置く
include/	プロジェクト固有のヘッダー（SystemData.h、ProjectConfig.h）
lib/	モジュール本体。1 モジュール = 1 フォルダ（.h + .cpp）
platformio.ini	ビルド設定ファイル（次の Q&A で詳しく解説）

▷ なぜこの構成なの？

PlatformIO の標準ルールとして、src/ にメインコード、lib/ にライブラリ（再利用可能なコード）、include/ にプロジェクト固有のヘッダーを置きます。Arduino IDE では全ファイルが 1 フォルダに並びますが、PlatformIO では役割ごとにフォルダが分かれているため、規模が大きくなっても見通しがよいままです。

Q: lib/ の中はどうなってるの？

lib/ には 2 種類のフォルダがあります。

7.1 コア層: ModuleCore/

IModule.h と ModuleTimer.h の 2 ファイルだけです。この 2 つはプロジェクト固有の型に一切依存していないので、別のプロジェクトにそのままコピーして使えます。

```
lib/ModuleCore/
  |-- IModule.h        ... インターフェース定義
  `-- ModuleTimer.h    ... ノンブロッキングタイマー
```

7.2 個別モジュール: XxxModule/

各ハードウェアに対応するモジュールが、それぞれ独立したフォルダに入っています。

```
lib/LedModule/
  |-- LedModule.h      ... Config/Data構造体 + クラス宣言
  `-- LedModule.cpp    ... メソッド実装
```

すべてのモジュールがこの「.h + .cpp」のペアで構成されています。新しいモジュールを追加するときも、同じ形式でフォルダを作るだけです。

! 1 モジュール = 1 フォルダを守る

PlatformIO の lib/は、フォルダ単位で独立したライブラリとして扱われます。1 つのフォルダに複数の無関係なモジュールを入れると、依存関係の管理が崩れます。必ず「1 モジュール = 1 フォルダ」を守ってください。

Q: include/ には何を置くの？

include/ には、プロジェクト全体で共有するヘッダーを置きます。本リポジトリでは 2 つだけです。

ファイル	役割
SystemData.h	全モジュールの Data 構造体を集約した SystemData 構造体を定義する
ProjectConfig.h	共有バスピン定義 + 全モジュールの Config インスタンスをまとめて定義する

この 2 ファイルが、個別モジュール（lib/）とメインコード（src/main.cpp）をつなぐ「接続点」です。

```
 lib/XxxModule/          include/            src/
+---------------+     +--------------+     +----------+
| XxxModule.h   |--->| SystemData.h |<---| main.cpp |
| XxxModule.cpp |     | ProjectConf  |     |          |
+---------------+     +--------------+     +----------+
```

モジュールを追加するときは、lib/にフォルダを作った後、この 2 ファイルにも追記するのを忘れないでください（実践の章で詳しく手順を示します）。

Q: なぜ PlatformIO を使うの？

本リポジトリでは開発環境に PlatformIO を使っています。Arduino IDE ではなく PlatformIO を選ぶ理由は、主に 3 つあります。

1. **環境の再現性**
   ボード設定・ライブラリバージョン・ビルドオプションがすべて platformio.ini に記述されるため、git clone するだけで誰でも同じ環境でビルドできます。
2. **ライブラリの分離管理**
   lib/ に置いたモジュールは、PlatformIO が自動でライブラリとして認識します。Arduino IDE のようにグローバルなライブラリフォルダを汚しません。
3. **VSCode の強力なエディタ機能**
   補完・定義ジャンプ・リファクタリングなど、大規模なコードを扱うときに欠かせない機能が使えます。

▷ Arduino IDE のコードはそのまま動く

PlatformIO は Arduino フレームワーク互換です。setup()、loop()、digitalWrite() など、Arduino IDE で書いたコードはほぼそのまま動きます。学習コストは「フォルダ構成の違いに慣れる」程度です。

Q: platformio.ini には何が書いてあるの？

platformio.ini は、プロジェクトのビルド設定を記述するファイルです。本リポジトリの設定を抜粋して解説します。

```
[env:esp32-s3-cam-n16r8]        ; 環境名（pio runで使う）
```

```
platform = espressif32          ; マイコンプラットフォーム
board = esp32-s3-devkitc-1      ; ボード種別
framework = arduino             ; Arduino互換フレームワーク
monitor_speed = 115200          ; シリアルモニターのボーレート

lib_deps =
    bodmer/TFT_eSPI @ ^2.5.43   ; 外部ライブラリ（バージョン指定）

build_flags =
    -D BOARD_HAS_PSRAM          ; コンパイルマクロ定義
    -I include                  ; include/ をインクルードパスに追加
    -I lib/ModuleCore           ; 各モジュールのパスも追加
    -I lib/LedModule
    ...
```

重要なポイントは build_flags の -I オプションです。

▷ なぜ -I でパスを追加するの？

PlatformIO の lib/内のコードから include/フォルダのヘッダーを使うには、インクルードパスの追加が必要です。-I include がないと、lib/LedModule/LedModule.cpp で #include "SystemData.h" と書いてもファイルが見つかりません。

ビルドと書き込みはターミナルから以下のコマンドで行います。

```
pio run                         # ビルド
pio run -t upload               # ビルド + ボードに書き込み
pio device monitor              # シリアルモニター
```

Q: ファイル同士の依存関係はどうなってるの？

ファイル間のインクルード関係を整理します。循環依存を防ぐためのルールがあります。

ファイル	インクルードするもの	ポイント
XxxModule.h	IModule.h	SystemData は前方宣言のみ
XxxModule.cpp	XxxModule.h, SystemData.h	ここで初めて SystemData の中身を使う
SystemData.h	各 XxxModule.h	Config/Data 構造体の定義を取得
ProjectConfig.h	各 XxxModule.h	Config インスタンスを定義
main.cpp	SystemData.h, ProjectConfig.h	すべてを組み合わせる

! ヘッダーで SystemData.h をインクルードしない

XxxModule.h で SystemData.h をインクルードすると、循環依存が発生します。SystemData.h が各モジュールの.h をインクルードしているため、お互いがお互いをインクルードする無限ループになってしまいます。ヘッダーでは必ず struct SystemData; の前方宣言だけにしてください。

8 実コードを読んでみよう

実践に入る前に、リポジトリに実装されている最もシンプルなモジュール「LedModule」の実コードを読んでみましょう。ここまで学んできたパターンが、実際のコードでどう使われているかを確認します。

Q: LedModule.h はどうなってるの？

まずはヘッダーファイル（lib/LedModule/LedModule.h）です。

```
// LedModule.h — LED制御モジュール
#pragma once
#include <Arduino.h>
#include "IModule.h"

// --- Config構造体 ---
struct LedConfig {
    uint8_t ledPin;  // LED出力ピン
};

// --- Data構造体 ---
struct LedData {
    bool state = false;
    uint8_t brightness = 0;
};

// SystemDataの前方宣言はIModule.h内で行われている

// --- モジュール実装 ---
class LedModule : public IModule {
private:
    LedConfig _config;

public:
    LedModule(const LedConfig& config);
    bool init() override;
    void updateOutput(SystemData& data) override;
};
```

ここまで学んだパターンがすべて揃っています。

- **Config 構造体**（第 3 章） — LedConfig にピン番号をまとめている
- **Data 構造体**（第 3 章） — LedData にデフォルト値が設定されている
- **IModule 継承**（第 4 章） — : public IModule で統一インターフェースを実装
- **前方宣言**（第 5 章） — SystemData.h をインクルードしていない（IModule.h の中で前方宣言済み）
- **override**（第 4 章） — 親クラスの仮想関数を上書きすることを明示
- **updateOutput**（第 5 章） — LED は出力モジュールなので updateOutput をオーバーライド

▷ ヘッダーには宣言だけ

メソッドの中身（実装）は書かれていません。「こういう関数がありますよ」という宣言だけです。実装は .cpp ファイルに書きます。これが第 7 章で説明した「.h + .cpp」の分離ルールです。

Q: LedModule.cpp はどうなってるの？

次に実装ファイル（lib/LedModule/LedModule.cpp）です。

```
// LedModule.cpp — LED制御モジュールの実装
#include "LedModule.h"
#include "SystemData.h"
#include <Arduino.h>

LedModule::LedModule(const LedConfig& config) : _config(config) {}

bool LedModule::init() {
    pinMode(_config.ledPin, OUTPUT);
    Serial.println("[Led] init OK");
    return true;
}

void LedModule::updateOutput(SystemData& data) {
    digitalWrite(_config.ledPin, data.led.state ? HIGH : LOW);
}
```

たった 16 行です。各メソッドを見ていきましょう。

8.1 コンストラクタ

```
LedModule::LedModule(const LedConfig& config) : _config(config) {}
```

Config を受け取って _config にコピーするだけです。ピン番号はハードコードせず、Config から取得します。

8.2 init()

```
bool LedModule::init() {
    pinMode(_config.ledPin, OUTPUT);
    Serial.println("[Led] init OK");
    return true;
}
```

_config.ledPin でピン番号にアクセスしています。ログフォーマットは **[モジュール名] メッセージ** の統一形式です。成功時は true を返します。

8.3 updateOutput()

```
void LedModule::updateOutput(SystemData& data) {
    digitalWrite(_config.ledPin, data.led.state ? HIGH : LOW);
}
```

SystemData から data.led.state を読み取り、LED の ON/OFF を制御しています。LED 自身は「なぜ光るのか」を知りません。ロジックフェーズが data.led.state を書き換え、LED モジュールはそれに従う

だけです。これが 3 フェーズモデルの「出力モジュール」の役割です。メソッド名が `updateOutput` なので、このモジュールが出力専用であることが一目で分かります。

▷ .cpp では SystemData.h をインクルード

`.h` では前方宣言だけでしたが、`.cpp` では `#include "SystemData.h"` でフル定義を取得しています。`data.led.state` のようにメンバにアクセスするには、SystemData の中身を知る必要があるためです。

Q: このモジュールはどうやって動くの？

LedModule が動くまでの全体の流れを確認しましょう。

1. `ProjectConfig.h` で設定を定義:

```
const LedConfig LED_CONFIG = { .ledPin = 2 };
```

2. `main.cpp` でインスタンスを作成し、配列に登録:

```
LedModule ledModule(LED_CONFIG);
IModule* outputModules[] = { &ledModule };
```

3. `setup()` で `init()` が呼ばれ、ピンが初期化される
4. `loop()` の出力フェーズで `updateOutput(systemData)` が呼ばれる
5. ロジックフェーズが `systemData.led.state` を書き換えると、次のループで LED に反映される

これが「Config → インスタンス化 → init → updateInput/updateOutput」の基本サイクルです。次の章では、このサイクルをゼロから自分で作ります。

9 実践 — センサーモジュールを作る

この章では、温度センサーモジュールを一から作ります。前章で実コードの読み方を確認したので、今度は自分で書いてみましょう。

Q: 新しいモジュールはどう作るの？

新しいモジュールを作る手順は、いつも同じ 7 ステップです。

1. Config 構造体を定義する
2. Data 構造体を定義する
3. クラスを宣言する（.h ファイル）
4. メソッドを実装する（.cpp ファイル）
5. SystemData に Data を追加する
6. ProjectConfig.h に Config インスタンスを追加する
7. main.cpp のモジュール配列に追加する

順番に見ていきましょう。analogRead() で温度を読む簡易センサーを題材にします。

9.1 ステップ 1: Config 構造体

起動時に決まる設定をまとめます。

```
// TempSensorModule.h 内で定義
struct TempSensorConfig {
    uint8_t analogPin;          // アナログ入力ピン
    float conversionFactor;     // ADC値→温度の変換係数
};
```

9.2 ステップ 2: Data 構造体

実行中に変化するデータをまとめます。デフォルト値を必ず設定します。

```
// TempSensorModule.h 内で定義
struct TempSensorData {
    float temperature = -999.0f;  // 温度（未取得時は異常値）
    bool isValid = false;         // データが有効か
};
```

▷ なぜデフォルト値を設定するの？

init() 前やセンサー故障時でも、SystemData の値は参照される可能性があります。デフォルト値がないと、不定値（ゴミデータ）が入ったまま使われてしまいます。`-999.0f` のような明らかにおかしい値を入れておけば、「まだ読めていない」ことが一目で分かります。

9.3 ステップ 3: クラス宣言（.h ファイル）

```
// TempSensorModule.h
#pragma once
#include <Arduino.h>
#include "IModule.h"
#include "ModuleTimer.h"

struct TempSensorConfig { /*（上で定義済み）*/ };
struct TempSensorData   { /*（上で定義済み）*/ };

struct SystemData;  // 前方宣言

class TempSensorModule : public IModule {
private:
    TempSensorConfig _config;
    ModuleTimer _readTimer;

public:
    TempSensorModule(const TempSensorConfig& config);
    bool init() override;
    void updateInput(SystemData& data) override;
};
```

ポイント:

- `#pragma once` — 二重インクルード防止
- `struct SystemData;` — 前方宣言（SystemData.h はインクルードしない）
- `: public IModule` — IModule を継承
- `override` — 親クラスの仮想関数を上書きすることを明示

9.4 ステップ 4: メソッド実装（.cpp ファイル）

```
// TempSensorModule.cpp
#include "TempSensorModule.h"
#include "SystemData.h"  // ここでフル定義をインクルード

TempSensorModule::TempSensorModule(const TempSensorConfig& config)
    : _config(config) {}

bool TempSensorModule::init() {
    // アナログピンはpinMode不要（Arduino標準動作）
    _readTimer.setTime();
    Serial.println("[TempSensor] init OK");
    return true;
}

void TempSensorModule::updateInput(SystemData& data) {
    // 1秒ごとに温度を読み取る
    if (_readTimer.getNowTime() < 1000) return;
    _readTimer.setTime();

    int raw = analogRead(_config.analogPin);
    data.temp.temperature = raw * _config.conversionFactor;
    data.temp.isValid = true;
}
```

9.5 ステップ 5: SystemData に追加

```
// SystemData.h
#include "LedModule.h"
#include "TempSensorModule.h"

struct SystemData {
    LedData led;
    TempSensorData temp;   // 追加！
};
```

9.6 ステップ 6: ProjectConfig.h に追加

```
// ProjectConfig.h
#include "TempSensorModule.h"
```

```
const TempSensorConfig TEMP_CONFIG = {
    .analogPin = 34,
    .conversionFactor = 3.3f / 4095.0f * 100.0f,
};
```

9.7 ステップ 7: main.cpp に追加

```
// main.cpp
TempSensorModule tempSensor(TEMP_CONFIG);

IModule* inputModules[] = {
    &tempSensor,   // 入力モジュール配列に追加
};
```

以上で完成です。新しいモジュールを追加するときも、同じ 7 ステップを繰り返すだけです。

Q: ModuleTimer はどう使うの？

組み込みプログラムでありがちなのが、delay() を使った待ち時間の実装です。

× delay() で待つ

```
void updateInput(SystemData& data) {
    int raw = analogRead(_config.analogPin);
    data.temp.temperature = raw * _config.conversionFactor;
    delay(1000);  // ここで1秒間、何もできない！
}
```

delay(1000) を呼ぶと、プログラム全体が 1 秒間停止します。その間、他のモジュールも動けません。LED の点滅もディスプレイの更新もすべて止まります。

✓ ModuleTimer で周期実行

```
class TempSensorModule : public IModule {
private:
    ModuleTimer _readTimer;  // タイマーをメンバ変数に

public:
    bool init() override {
        _readTimer.setTime();  // 現在時刻を記録
        return true;
    }

    void updateInput(SystemData& data) override {
        // 前回から1秒経っていなければ即リターン
        if (_readTimer.getNowTime() < 1000) return;
        _readTimer.setTime();  // 時刻をリセット
```

```
15
16          // 1秒経ったときだけ実行
17          int raw = analogRead(_config.analogPin);
18          data.temp.temperature = raw * _config.conversionFactor;
19          data.temp.isValid = true;
20      }
21  };
```

ModuleTimer は `millis()` をラップした軽量なタイマーです。

- `setTime()` — 現在時刻を記録する
- `getNowTime()` — 前回の setTime() からの経過ミリ秒を返す

delay() と違い、プログラムをブロックしません。経過時間が足りなければ即座に return し、他のモジュールに処理を譲ります。

! delay() は原則禁止

3 フェーズモデルでは、各モジュールの `updateInput()`/`updateOutput()` はできるだけ速く返る必要があります。`delay()` を使うと他のすべてのモジュールが影響を受けるため、ModuleTimer を使ったノンブロッキング方式を徹底してください。

10 設計のコツ集

この章では、実装でよくある疑問に短い Q&A 形式で答えます。

Q: エラーが起きたらどうする？

`init()` は `bool` を返します。失敗したら `false` を返しましょう。

```
1  bool TempSensorModule::init() {
2      if (!checkSensor()) {
3          Serial.println("[TempSensor] init failed: sensor not found");
4          return false;  // 失敗を通知
5      }
6      return true;
7  }
```

`updateInput()`/`updateOutput()` でエラーが起きた場合は、Data のフラグで通知します。

```
1  void TempSensorModule::updateInput(SystemData& data) {
2      int raw = analogRead(_config.analogPin);
3      if (raw < 0) {
4          data.temp.isValid = false;  // エラーフラグ
5          return;
6      }
7      data.temp.temperature = raw * _config.conversionFactor;
8      data.temp.isValid = true;
```

```
}
```

Q: SPI や I2C を複数モジュールで共有するには？

バス（SPI、I2C）のインスタンスは main.cpp で作り、コンストラクタの引数でモジュールに渡します。

```
// main.cpp
static TwoWire i2cBus = TwoWire(0);

void setup() {
    i2cBus.begin(SDA_PIN, SCL_PIN);  // バス初期化は先に！

    // モジュール初期化は後
    tempSensor.init();
    display.init();
}
```

```
// モジュール側 — ポインタで受け取る
class TempSensorModule : public IModule {
private:
    TwoWire* _wire;

public:
    TempSensorModule(const TempSensorConfig& config,
                     TwoWire* wire)
        : _config(config), _wire(wire) {}
};
```

！バス初期化の順番

bus.begin() は必ず setup() 内で、全モジュールの init() より前に呼んでください。モジュールの init() 内で bus.begin() を呼ぶと、2 つ目のモジュールが初期化するときにバスがリセットされる可能性があります。

SPI 使用時は排他制御を忘れないでください。

```
void updateOutput(SystemData& data) {
    _spi->beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0));
    // SPI通信
    _spi->endTransaction();
}
```

Q: init 失敗したモジュールは復帰できる？

できます。ロジックフェーズで定期的に init() を再試行します。

```
// ロジックフェーズ内
void applyPattern(SystemData& data) {
```

```
    // 無効化されたモジュールの再init（10秒ごと）
    static ModuleTimer retryTimer;
    if (retryTimer.getNowTime() >= 10000) {
        retryTimer.setTime();
        for (int i = 0; i < INPUT_COUNT; i++) {
            if (!inputModules[i]->enabled) {
                if (inputModules[i]->init()) {
                    inputModules[i]->enabled = true;
                    Serial.println("[System] module recovered");
                }
            }
        }
    }
}
```

接触不良で一時的に通信が切れたセンサーが、再接続後に自動復帰します。

Q: ログはどう書くのがいい？

統一フォーマットを使いましょう。

```
// フォーマット: [モジュール名] メッセージ
Serial.println("[TempSensor] init OK");
Serial.println("[TempSensor] read failed: timeout");
Serial.println("[LED] state changed: ON");
```

角括弧でモジュール名を付けると、シリアルモニターでどのモジュールの出力か一目で分かります。

Q: 実行中に Config を変更したいときは？

Config 変更はロジックフェーズでのみ行います。

× 入力フェーズで Config 変更

```
// BLEモジュールのupdateInput()内で直接変更 → 危険！
void BleModule::updateInput(SystemData& data) {
    if (newConfigReceived) {
        ledModule._config.pin = newPin;  // 他モジュールを直接操作
    }
}
```

✓ SystemData 経由でロジックフェーズに渡す

```
// BLEモジュール — 受信データをSystemDataに書くだけ
void BleModule::updateInput(SystemData& data) {
    if (newConfigReceived) {
        data.ble.requestedLedPin = newPin;
        data.ble.configUpdateRequested = true;
    }
```

```
}

// ロジックフェーズ — ここでConfig変更を実行
void applyPattern(SystemData& data) {
    if (data.ble.configUpdateRequested) {
        // 安全にConfig変更を適用
        data.ble.configUpdateRequested = false;
    }
}
```

Q: ビルドが通らない！よくあるエラーは？

初めてモジュールを追加したとき、コンパイルエラーに悩むことが多いです。よくあるエラーとその対処法をまとめます。

10.1 「No such file or directory」

```
fatal error: SystemData.h: No such file or directory
```

lib/ 内のコードから include/ のヘッダーが見つかっていません。platformio.ini の build_flags に -I include が入っているか確認してください。

```
build_flags =
    -I include              ; これがないとlib/からinclude/が見えない
```

10.2 「does not name a type」

```
error: 'SystemData' does not name a type
```

ヘッダーファイルで struct SystemData; の前方宣言を忘れている、または .h で SystemData.h をインクルードしてしまい循環依存が発生しています。以下を確認してください。

- XxxModule.h — struct SystemData; の前方宣言があるか
- XxxModule.h — SystemData.h をインクルードしていないか
- XxxModule.cpp — #include "SystemData.h" があるか

10.3 「is not a member of 'SystemData'」

```
error: 'struct SystemData' has no member named 'temp'
```

新しいモジュールの Data を SystemData.h に追加し忘れています。include/SystemData.h を開いて、自分のモジュールの #include と Data メンバを追加してください（実践の章ステップ 5 参照）。

10.4 「marked 'override', but does not override」

```
error: 'bool XxxModule::init()' marked 'override', but does not
override any member functions
```

IModule を継承していないか、メソッドのシグネチャ（引数・戻り値）が合っていません。: public IModule の継承と、init()/updateInput()/updateOutput() の引数が正しいか確認してください。

▷ エラーメッセージの読み方

コンパイルエラーは上から順に読みます。最初のエラーが原因で後続のエラーが連鎖的に発生することが多いため、まず最初の 1 つを修正してから再ビルドしてください。

Q: deinit() はいつ使うの？

deinit() は、モジュールが確保したリソースを解放するためのメソッドです。IModule にデフォルトの空実装があるので、必要なモジュールだけオーバーライドします。

```
class CameraModule : public IModule {
public:
    void deinit() override {
        // フレームバッファを解放
        if (_fb) {
            esp_camera_fb_return(_fb);
            _fb = nullptr;
        }
        esp_camera_deinit();
        Serial.println("[Camera] deinit done");
    }
};
```

主な用途は以下の通りです。

- ディープスリープに入る前のリソース解放
- モジュールの動的な停止・再起動
- テスト時の後片付け

付録 A: PlatformIO のススメ

この付録では、本書のサンプルプロジェクトで使用している開発環境「PlatformIO」について解説します。

Q: PlatformIO って何？

PlatformIO は、マイコンのプログラムを書いてボードに書き込むための開発環境です。Arduino IDE と同じ役割を果たしますが、大きな違いが 1 つあります。

- **Arduino IDE** — マイコン開発専用のスタンドアロンアプリ

- **PlatformIO** — VSCode（Visual Studio Code）の拡張機能として動作する

Arduino 互換なので、Arduino IDE で書いたコードはほぼそのまま PlatformIO でも動きます。「Arduino IDE に慣れているからコードが書けない」という心配は不要です。

▷ VSCode とは

VSCode（Visual Studio Code）は Microsoft が無料で提供する高機能テキストエディタです。拡張機能を追加することでさまざまな言語・用途に対応でき、現在最も広く使われている開発エディタの一つです。

Q: Arduino IDE と何が違うの？

主な違いを比較表にまとめます。

項目	Arduino IDE	PlatformIO
エディタ	専用エディタ（機能が限定的）	VSCode（補完・定義ジャンプ等が強力）
ライブラリ管理	グローバルに 1 バージョンのみ	プロジェクトごとにバージョン固定
ボード設定	GUI で毎回選択	`platformio.ini` に記述（固定）
環境の共有	口頭やメモで伝える	`platformio.ini` を Git で共有
CI/CD	対応困難	CLI ベースで容易
複数ボード対応	手動で切り替え	`[env:名前]` で同時管理

特に「ライブラリのバージョン管理」と「環境の共有」が PlatformIO の大きな強みです。

Q: チーム開発で PlatformIO が強い理由は？

Arduino IDE でチーム開発をしようとすると、よくこんな問題が起きます。

× Arduino IDE でのチーム開発あるある

- 「ボードマネージャ v2.0.11 じゃないとビルド通らないんだけど…」
- 「ライブラリのバージョン合わせて」→ 口頭伝達 → 忘れる → ビルドが通らない
- 「え、自分の PC では動くんだけどな」

PlatformIO なら、プロジェクトに必要な情報がすべて `platformio.ini` に記述されています。

✓ PlatformIO なら git clone するだけで同じ環境が再現される

```
git clone https://github.com/yourname/yourproject
cd yourproject
pio run          # 同じplatformio.iniからビルド → 全員同じ結果
```

▷ AI とのデバッグでも強い

ChatGPT や Claude などの AI ツールにデバッグを相談するとき、platformio.ini を貼り付けるだけでプラットフォーム・ボード・ライブラリバージョンが確定した状態で相談できます。「環境が違うのでは？」という的外れな回答が減ります。

Q: platformio.ini の読み方

platformio.ini は、プロジェクトのビルド設定をすべて記述するファイルです。本リポジトリの設定を例に各項目を解説します。

```
[env:esp32-s3-cam-n16r8]    ; ビルド対象の環境定義（名前は自由）
platform = espressif32      ; 使用するマイコンプラットフォーム
board = esp32-s3-devkitc-1  ; 対象ボード
framework = arduino         ; Arduinoフレームワークを使用
monitor_speed = 115200      ; シリアルモニターのボーレート

lib_deps =
    bodmer/TFT_eSPI @ ^2.5.43  ; 使用ライブラリとバージョン

build_flags =
    -D BOARD_HAS_PSRAM     ; コンパイルマクロの定義
    -I include             ; include/ フォルダをパスに追加
```

各項目の役割をまとめます。

項目	説明
[env: 名前]	ビルド対象の環境定義。複数書けば複数ボードを管理できる
platform	使用するマイコンプラットフォーム（例: espressif32）
board	対象ボードの識別子
framework	arduino または espidf 等
lib_deps	使用ライブラリとバージョン。@^2.5.43 はマイナーバージョン以上を許容
build_flags	コンパイルオプション（マクロ定義・インクルードパス等）
monitor_speed	シリアルモニターのボーレート

▷ Arduino IDE との対応

Arduino IDE で言うところの「ボードマネージャの URL 追加 → ボード選択 → ライブラリマネージャでインストール」を、この platformio.ini 1 ファイルで全部済ませています。初回 pio run 時に必要なものが自動でダウンロードされます。

Q: PlatformIO の始め方

以下のステップで環境を構築できます。

1. **VSCode をインストール**
   https://code.visualstudio.com/ からダウンロードしてインストールする
2. **拡張機能「PlatformIO IDE」をインストール**
   VSCode の拡張機能タブ（`Ctrl+Shift+X`）で「PlatformIO IDE」を検索してインストールする
3. **プロジェクトを開く**
   PlatformIO のホーム画面から「Open Project」で既存プロジェクトを開く、
   または「New Project」でボードを選んで新規作成する
4. **ビルドする**
   画面下のチェックマーク（✓）をクリック、またはターミナルで `pio run`
5. **書き込む**
   画面下の矢印（→）をクリック、またはターミナルで `pio run -t upload`
6. **シリアルモニターを開く**
   画面下のコンセントマークをクリック、またはターミナルで `pio device monitor`

▷ 本書のサンプルプロジェクトについて

本書のサンプルプロジェクトも PlatformIO 形式です。`git clone` して VSCode で開くだけで、すぐにビルド・書き込みができます。ライブラリのインストール等は `platformio.ini` が自動で処理します。